Test Plan – C.R.A.M.

Change Log

Version	Change Date	By	Description
1.0	March 6, 2026	Daniyal Hasnain	Initial Testing Plan
2.0	March 25, 2026	Sam Iezzi	Expanded Plan
3.0	April 3, 2026	Sam Iezzi	Expanded Load Tests

1 Introduction

1.1 Scope

Three of the five features were completed and unit tested.

1. Account Creation
2. Course Provisioning
3. Section Management
4. Global Search
5. Profile Management

Account Creation

Account creation will be tested to verify that valid inputs (non-empty username, a valid university email domain, and a password of at least 8 characters) result in a new user being stored and a 201 response, and that invalid or duplicate inputs are correctly rejected with the appropriate HTTP error codes (403, 409, 422, 500).

Course Provisioning

Course provisioning will be tested to verify that courses are correctly fetched and rendered, that loading and error states are handled gracefully, that search filtering returns only matching results by title or course code, and that helper functions correctly transform course data for display and routing.

Section Management

Section management will be tested to verify that valid inputs result in a section being created, updated, or deleted and that the appropriate response is returned, and that unauthenticated requests and invalid or missing data are correctly rejected with the appropriate HTTP error codes (401, 404, 409, 422, 500). The same is verified for definitions, which share the same CRUD structure and error handling.

Global Search

Global search will be tested to verify that courses and sections are correctly fetched and navigated, that loading and error states are handled gracefully, that search filtering returns only matching results by title, description and body for sections, or title and course code for courses.

Profile Management

Profile management will be tested to verify that valid inputs (non-empty username, a valid university email domain, and a password of at least 8 characters) will successfully result in profile management features (such as reset password and verify email) success and a 201 response, and that invalid or duplicate inputs are correctly rejected with the appropriate HTTP error codes (403, 409, 422, 500).

1.2 Roles and Responsibilities

The following roles were applied across the team members. A single member may hold multiple roles.

Frontend Developer: Responsible for the React web application's UI and UX, including component design, user interactions, and sending data to the backend via API requests.

Backend Developer: Responsible for receiving requests from the frontend, processing them, and storing or retrieving data from MongoDB.

Database Manager: Responsible for setting up MongoDB, defining the data schemas, and managing the database structure.

QA Analyst: Responsible for writing and maintaining unit tests to verify the correctness of frontend and backend logic.

DevOps: Responsible for setting up and maintaining the CI pipeline to automate test execution on each push to the main and development branch.

Name	Net ID	GitHub Username	Role
Shivam	shivam	Shivambhagat20	DevOps Database Manager QA Analyst
Adam Rossow	rossowa	rossowadam	Backend Developer Frontend Developer
Sam Iezzi	iezzis	Iezzi-S	Database Manager Backend Developer QA Analyst
Daniyal Hasnain	hasnaind	daniyal-h	Backend Developer Frontend Developer QA Analyst
David Schwarz	schwarzd	schwarzd-dev	Frontend Developer Backend Developer

The project was more issue-based and saw members assuming different roles to accomplish their tasks, as opposed to remaining within the bounds of a single role.

2 Test Methodology

2.1 Automated Regression Testing

2.1.1 Test Levels

The Sprint 3 test plan covered unit tests across the three implemented features, numbering at 71 backend tests and 227 frontend tests, alongside infrastructure steps through GitHub Actions.

Test Level	Scope & Requirement
Unit Testing	At least **10** tests per feature for a total of **298** tests.
Integration Testing	**14** tests covering interactions between features.
Acceptance Testing	**End-user testing** for every user story.
Regression Testing	Unit & Integration tests executed on **every push** to the **Main and Development** branches.

Unit Testing Methodology: Vitest was used for all unit tests. Frontend tests use React Testing Library to render components in a jsdom environment with mocked API modules and stubbed child components. Backend tests mock fetch responses directly to verify correct handling of success and error status codes. Pure helper functions are tested in isolation with no mocking required.

Regression Testing Methodology: A GitHub Actions CI pipeline is configured to trigger on every push to the main and development branches.

Integration Testing Methodology: Integration tests were completed using supertest and Node's build in testing framework. Integration tests mock the app listener and created a testing database to improve test performance and prevent testing artifacts from remaining in the production database. Integration tests cover all API pathways.

Acceptance Testing Methodology: Team will walkthrough user stories. **A comprehensive write up** going through the steps for each user story will be included in a **separate document** within the documents folder.

2.1.2 CI/CD Regression Workflow

Every time a commit is made or a pull request is created to the Main or Development branches, it triggers GitHub Actions to run all the unit tests and infrastructure steps including environment

setup, dependency installation, and lint checks. Squashing to Main is blocked if any of these pipelining tests fail.

2.2 Mutation Testing

Tool: Stryker Mutator

Backend Summary

Results:

- **Total Mutants:** 725
- **Surviving Mutants:** 0
- **Timeouts:** 81
- **Mutation Score:** 100%

Key Takeaway:

The test suite achieved **100% mutation coverage**, successfully killing all 725 generated mutants. While 81 mutants resulted in timeouts (indicating potential infinite loops or heavy resource locks caused by the mutations), the zero survival rate confirms a highly robust and effective test suite.

Frontend Summary

Results:

- **Total Mutants:** 408
- **Surviving Mutants:** 0
- **Compile Errors:** 52
- **Mutation Score:** 100%

The test suite achieved **100% mutation coverage**, successfully killing all 408 generated mutants. Killing these mutants required a massive expansion of the frontend test suite, leading to the CRAM frontend now having a versatile testing suite.

2.3 Load Testing

Tool: Artillery

All relevant files located in /CRAM/backend/tests/load/. The script to run it is npm run test:load.

One test was conducted in sprint 3, tested using a local test DB. This tested the application logic's limitations. Two new tests were conducted in sprint 4, using a test folder in our real DB. This tested the limits of the entire system flow. One were the server was running on my machine, the other running in the Docker container.

Summary - Sprint 3

The test was configured to run for 60 seconds creating 10 new virtual users each second.

Test Overview

- **Throughput:** 110 requests/sec (Avg)
- **Total Requests:** 7,768
- **Total Virtual Users (VUs):** 600 created / 512 completed

The system demonstrates high performance for the majority of users but shows some instability under peak load. With a **median response time of 7ms**, while the slowest 5% of users experience delays exceeding **5.4 seconds**.

Critical Metrics

- **Success Rate:** ~85% (512 of 600 VUs completed successfully).
- **Error Rate:** 1.1% of total requests failed specifically due to ETIMEDOUT
- **Latency Distribution:**
    - **Median:** 7ms.
    - **Mean:** 650ms.
    - **Bottom 5%:** 5,487ms.
    - **Bottom 1%:** 9,047ms.

Analysis & Bottlenecks

The median response time of 7 ms shows that the system can handle an overwhelming majority of requests under load quite well. While there does exist some bottlenecking with 5% of responses taking over 5000ms to complete.

Summary - Sprint 4 local machine test

The test was configured to run for 30 seconds creating 10 new virtual users each second. With a one second delay between each request in the test script, to more realistically reflect real users. The changes were made to account for the constraints of our MongoDb access tier.

Test Overview

- **Throughput:** 60 requests/sec (Avg)
- **Total Requests:** 4354
- **Total Virtual Users (VUs):** 300 created / 285 completed

This test shows the constraints of MongoDB, with thousands of requests reaching out to the DB we saw a **median response time of 1826 ms**, while the slowest 5% of users experience delays exceeding **6976 ms**.

Critical Metrics

- **Success Rate:** ~95% (285 of 300 VUs completed successfully).
- **Error Rate:** 4.3% of total requests failed due to ETIMEDOUT, 0.6% had created duplicates and received
- **Latency Distribution:**
    - **Median:** 1826 ms.
    - **Mean:** 2295 ms.
    - **Bottom 5%:** 6976ms.
    - **Bottom 1%:** 9416ms.

Analysis & Bottlenecks

The changes in results compared with the sprint 3 test show that the limitations with CRAM are a result of the restrictions of our MongoDB access level, and not with our system. The high response times show that there is a bottleneck in the DB operations. Were this to become a full release, upgrading to higher tier would lead to faster performance, and be a necessity.

Summary - Sprint 4 container test

The test was configured to run for 30 seconds creating 10 new virtual users each second. With a one second delay between each request in the test script, to more realistically reflect real users. The changes were made to account for the constraints of our MongoDb access tier.

Test Overview

- **Throughput:** 49 requests/sec (Avg)
- **Total Requests:** 3956
- **Total Virtual Users (VUs):** 300 created / 260completed

This test shows the constraints of MongoDB, with thousands of requests reaching out to the DB we saw a **median response time of 1826 ms**, while the slowest 5% of users experience delays exceeding **6976 ms**.

Critical Metrics

- **Success Rate:** ~87% (260 of 300 VUs completed successfully).
- **Error Rate:** 12.3% of total requests failed due to ETIMEDOUT, 0.6% had created duplicates and received
- **Latency Distribution:**
    - **Median:** 889 ms.
    - **Mean:** 1464 ms.
    - **Bottom 5%:** 5065 ms.
    - **Bottom 1%:** 8868 ms.

Analysis & Bottlenecks

With the server running in the Docker container the test shows a significant boost in speed, with the median and mean response times dropping by almost 1 whole second. Although there is a bit more bottlenecking, with an increased failure rate of 12.7%

3 Abbreviations

Abbreviation	Definition
API	Application Programming Interface
AUT	Application Under Test
Auth	Authentication
CI	Continuous Integration
CD	Continuous Deployment
CRAM	Collaborative Resource for Academic Materials
CRUD	Create, Read, Update, and Delete
Config	Configuration
CSS	Cascading Style Sheets
Docs	Documentations
Feat	Feature
Git	Git Version Control
HTML	Hypertext Markup Language
ID	Identification
JS	JavaScript
JSON	JavaScript Object Notation
JWT	JSON Web Token
lib	Libraries
MD	Markdown
msg	Message
Ref	Reference
RTE	Rich Text Editor

SCSS	Sassy CSS
src	Source
SVG	Scalable Vector Graphics
TS	TypeScript
TSX	TypeScript XML/JSX
UI	User Interface
utils	Utilities
UX	User Experience
YAML	YAML Ain’t Markup Language
XML	Extensible Markup Language